

Closing the Triage Loop

MCP-mediated KLEE validation of
LLM-reported C/C++ vulnerabilities

Christopher Scherb, Alexander Trapp, Ruben Hutter,
Nico Bachmann, Luc Bryan Heitz

University of Applied Sciences and Arts Northwestern Switzerland

KES 2026 | **Dublin**

curl: AI reports created real work for maintainers

< 5%

of security submissions
confirmed as vulnerabilities
in 2025

- ▶ Low-quality AI reports consumed maintainer time
- ▶ curl ended bounty rewards on **31 January 2026**
- ▶ By April, AI-assisted reports had improved in quality; volume was higher

A convincing report still needs independent verification

Daniel Stenberg: The end of the curl bug-bounty (26 Jan 2026); High-Quality Chaos (22 Apr 2026).

The problem: which AI findings are real?

**An LLM flags a possible C/C++ bug.
Would you report it?**

- ▶ We need a reproducible check for each suspected error
- ▶ A failure may depend on inputs that real callers never provide
- ▶ Manually building a test and checking callers takes work

Our goal: check each candidate and explain the conditions under which it can fail

The LLM selects what to check

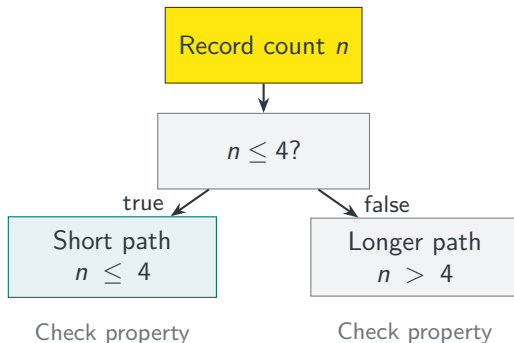
- ▶ Read the code and identify a suspicious function
- ▶ Identify inputs that should vary during analysis
- ▶ Inspect callers and propose assumptions about valid inputs
- ▶ Submit a structured candidate to the verification tool



The LLM supplies a hypothesis and context; execution must check it

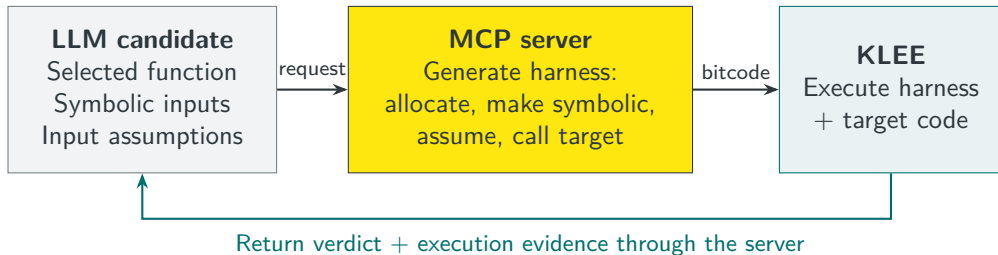
Symbolic execution checks possible executions

- ▶ Use variables instead of one fixed test input
- ▶ Follow branches and collect their conditions
- ▶ Solve those conditions to find possible paths
- ▶ If a path fails, obtain a concrete input that reaches it



KLEE is the symbolic execution engine used in our system

From the LLM candidate to a KLEE verdict



- ▶ **MCP** (Model Context Protocol) connects the LLM to the server
- ▶ A **harness** sets up symbolic inputs and calls the target function
- ▶ The server compiles harness + target to LLVM bitcode and runs KLEE
- ▶ The result returns to the LLM for interpretation and caller review

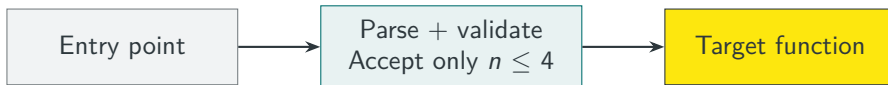
What the verdicts mean

Confirmed	A matching error was found; a concrete input demonstrates it
Infeasible	No matching error exists in the fully explored input model
Unresolved	The run timed out, failed to build, or hit an engine error

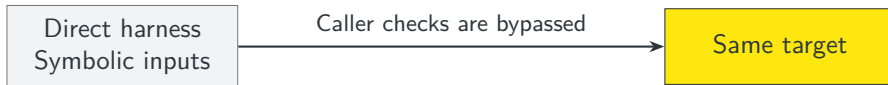
Each verdict applies to the supplied harness and input model

Where do we start symbolic execution?

START FROM AN APPLICATION / API ENTRY



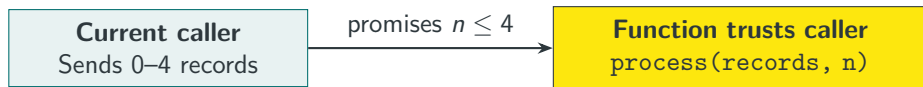
START DIRECTLY AT THE FUNCTION



- ▶ **From an entry point:** preserve caller checks; often a larger search
- ▶ **Directly at the function:** focus the search, but lose caller guarantees

Our direct harness needs assumptions about the missing caller context

A caller contract is a promise about inputs



- ▶ Model the promise in the harness: assume $0 \leq n \leq 4$
- ▶ Exclude $n > 4$ paths; fewer loop iterations reduce the search
- ▶ A harness assumption adds no runtime check to the real function

libpng examples

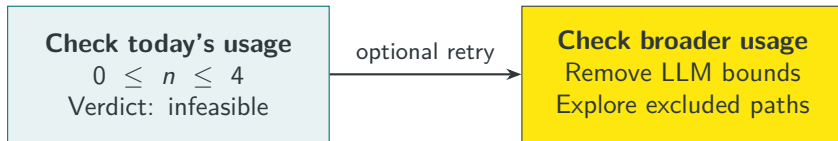
Why caller context changes the interpretation

CVE-2015-8540 helper
png_format_number

Local error without the contract; infeasible with it
Local error; all reviewed callers satisfy its precondition

A local error need not be reachable through real callers

Why retry: what if the function is used differently?



Possible results: an error appears; no error is found after completion; or the larger search remains unresolved.

- ▶ The LLM may have inferred a bound that is too restrictive
- ▶ A new caller or refactor may pass more records, such as $n = 8$
- ▶ Retry at the same function, without the LLM bounds

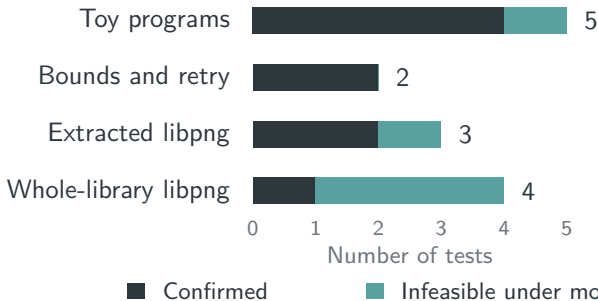
A new error can reveal a fragile assumption, even if today's usage is safe

What we tested

- ▶ **Small C programs:** buffer overflow, integer overflow, null-pointer dereference, use-after-free, and a safe example
- ▶ **Input bounds and retry:** check behavior under different assumptions
- ▶ **Extracted libpng functions:** check individual helpers
- ▶ **Linked libpng + zlib:** check functions with library code available

- ▶ KLEE 3.1; LLVM/Clang 13; STP + MiniSat
- ▶ Whole-library target: 2.2 MB bitcode, 30 translation units

All 14 tests returned the expected verdict



- ▶ 9 confirmed
- ▶ 5 infeasible
- ▶ 0.9–42.9 s per test

Evidence of internal consistency on these selected cases

- ▶ External precision, recall, and analyst time savings remain unmeasured

With an input bound, this libpng check finishes

- ▶ Same libpng number parser and harness; only the input bound changes

Without the input bound

60 seconds

Time limit reached
Paths still unchecked
Verdict: unresolved

With $\text{size} \leq 4$

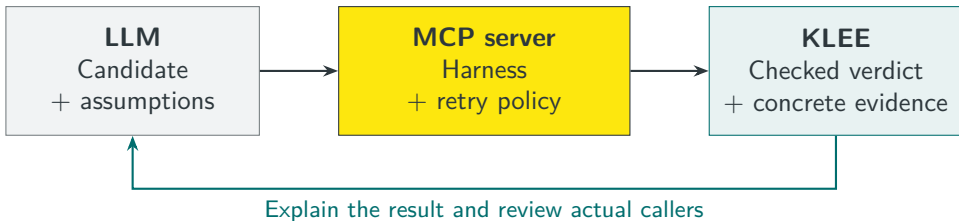
30.9 seconds

All modeled paths checked
No matching error found
Verdict: infeasible

- ▶ Restricting size to at most four excludes part of the search

The completed check covers only inputs allowed by the bound

Conclusion: the LLM guides, KLEE checks



- ▶ A tool interface turns a candidate into an executable check
- ▶ The verdict and its assumptions make the finding reviewable
- ▶ Next: evaluate independent real reports and analyst time savings

Would you report it now? Check the finding. Explain its scope.

github.com/FHNW-Security-Lab/klee-mcp

Questions?

Backup: the benchmark rows

Candidate	Verdict	Wall time (s)
bof_01	confirmed	1.0
intoverflow_01	confirmed	2.4
null_01	confirmed	0.9
uaf_01	confirmed	0.9
safe_01	infeasible	1.1
bof_01_bounded	confirmed	0.9
bof_01_tootight	confirmed after retry	0.9
libpng_fp_number	infeasible	30.9
libpng_fp_number_bug	confirmed	5.9
libpng_fp_number_unb.	confirmed	9.4
libpng_sig_cmp	infeasible	5.1
libpng_fp_whole	infeasible	42.9
libpng_push_read	infeasible	4.1
libpng_format_number	confirmed; contract violation	4.2

Infeasible verdicts are conditional on the harness and engine model.

Backup: larger targets expose model limits

Target	Run result	Interpretation
libpng tRNS	240 s; timeout	Analysis unresolved
libjpeg-turbo	5.4 s; infeasible 1 completed path	Error handling limits exploration; weak evidence of safety
libxml2	23 s; infeasible 11 completed paths	A second whole-library run; still only the modeled inputs

- ▶ Whole-library linking is not whole-program coverage
- ▶ Read path statistics and unsupported behavior with the verdict

Backup: limitations and next steps

- ▶ **Small benchmark:** evaluate an independent set of real findings
- ▶ **Engine limits:** improve handling of complex parser error paths
- ▶ **Caller assumptions:** validate them against realistic entry points
- ▶ **Impact assessment:** review automated labels before reporting

A local failure is one step toward a useful security report

Backup: further reading

- ▶ Cadar, Dunbar, Engler (2008)
KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs, OSDI
- ▶ Ramos and Engler (2015)
Under-Constrained Symbolic Execution: Correctness Checking for Real Code, USENIX Security

Project: github.com/FHNW-Security-Lab/klee-mcp